Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине: «Интеллектуальный анализ данных»
Тема «PCA»

Выполнил:
Студент 4 курса
Группы ИИ-26
Вирко Е.Д.
Проверил:
Никитин С.К.

Брест 2026

Цель: научиться применять метод PCA для осуществления визуализации данных

Задание на выполнение

1. Используя выборку по варианту, осуществить проецирование данных на плоскость первых двух и трех главных компонент (двумя способами: 1. вручную через использование numpy.linalg.eig для вычисления собственных значений и собственных векторов и 2. с помощью sklearn.decomposition.PCA для непосредственного применения метода PCA – два независимых варианта решения);
2. Выполнить визуализацию полученных главных компонент с использованием средств библиотеки matplotlib, обозначая экземпляры разных классов с использованием разных цветовых маркеров;
3. Используя собственные значения, рассчитанные на этапе 1, вычислить потери, связанные с преобразованием по методу PCA. Сделать выводы;
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Вариант 1

№ варианта	Выборка	Класс
1	seeds.zip	Последняя колонка

Ход работы

Код программы:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

df = pd.read_csv('seeds.zip', compression='zip', sep=r'\s+', header=None)

X = df.iloc[:, :-1]
y = df.iloc[:, -1]

X = X.fillna(X.mean())

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

cov_matrix = np.cov(X_scaled, rowvar=False)

eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)

idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]

X_pca_manual_2 = np.real(X_scaled.dot(eigenvectors[:, :2]))
X_pca_manual_3 = np.real(X_scaled.dot(eigenvectors[:, :3]))

pca_2 = PCA(n_components=2)
X_pca_sklearn_2 = pca_2.fit_transform(X_scaled)

pca_3 = PCA(n_components=3)
X_pca_sklearn_3 = pca_3.fit_transform(X_scaled)

classes = y.unique()
hex_colors = ['#440154', '#21918c', '#fde725', '#3b528b', '#5ec962']

fig1, (ax_a, ax_b) = plt.subplots(1, 2, figsize=(12, 5))
```

```
for i, cls in enumerate(classes):
    mask = (y == cls)
    ax_a.scatter(X_pca_manual_2[mask, 0], X_pca_manual_2[mask, 1],
                 label=f'Класс {cls}', color=hex_colors[i], alpha=0.7)
ax_a.set_title('2D PCA (numpy.linalg.eigh)')
ax_a.set_xlabel('Главная компонента 1')
ax_a.set_ylabel('Главная компонента 2')
ax_a.legend()
ax_a.grid(True)

for i, cls in enumerate(classes):
    mask = (y == cls)
    ax_b.scatter(X_pca_sklearn_2[mask, 0], X_pca_sklearn_2[mask, 1],
                 label=f'Класс {cls}', color=hex_colors[i], alpha=0.7)
ax_b.set_title('2D PCA (sklearn)')
ax_b.set_xlabel('Главная компонента 1')
ax_b.set_ylabel('Главная компонента 2')
ax_b.legend()
ax_b.grid(True)

fig1.tight_layout()
fig1.savefig('pca_2d.png', dpi=300, bbox_inches='tight')

fig2 = plt.figure(figsize=(14, 6))

ax1 = fig2.add_subplot(121, projection='3d')
for i, cls in enumerate(classes):
    mask = (y == cls)
    ax1.scatter(X_pca_manual_3[mask, 0], X_pca_manual_3[mask, 1], X_pca_manual_3[mask, 2],
                label=f'Класс {cls}', color=hex_colors[i])
ax1.set_title('3D PCA (numpy.linalg.eigh)')
ax1.set_xlabel('ГК 1')
ax1.set_ylabel('ГК 2')
ax1.set_zlabel('ГК 3')
ax1.legend()

ax2 = fig2.add_subplot(122, projection='3d')
for i, cls in enumerate(classes):
    mask = (y == cls)
    ax2.scatter(X_pca_sklearn_3[mask, 0], X_pca_sklearn_3[mask, 1], X_pca_sklearn_3[mask, 2],
                label=f'Класс {cls}', color=hex_colors[i])
ax2.set_title('3D PCA (sklearn)')
ax2.set_xlabel('ГК 1')
ax2.set_ylabel('ГК 2')
ax2.set_zlabel('ГК 3')
ax2.legend()

fig2.tight_layout()
fig2.savefig('pca_3d.png', dpi=300, bbox_inches='tight')

plt.close('all')

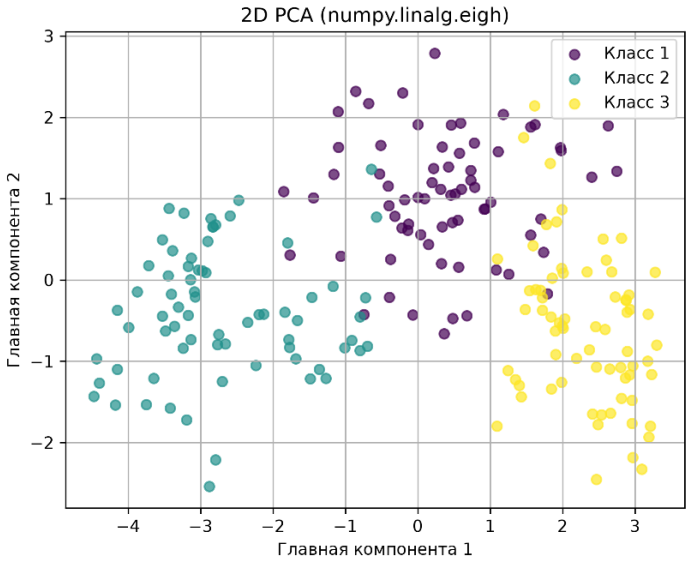
total_variance = sum(eigenvalues)

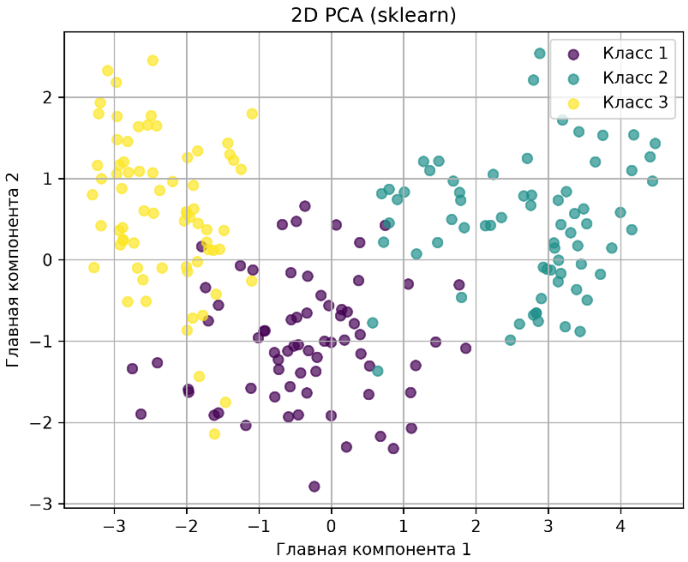
loss_2 = 1 - (sum(eigenvalues[:2]) / total_variance)
loss_3 = 1 - (sum(eigenvalues[:3]) / total_variance)

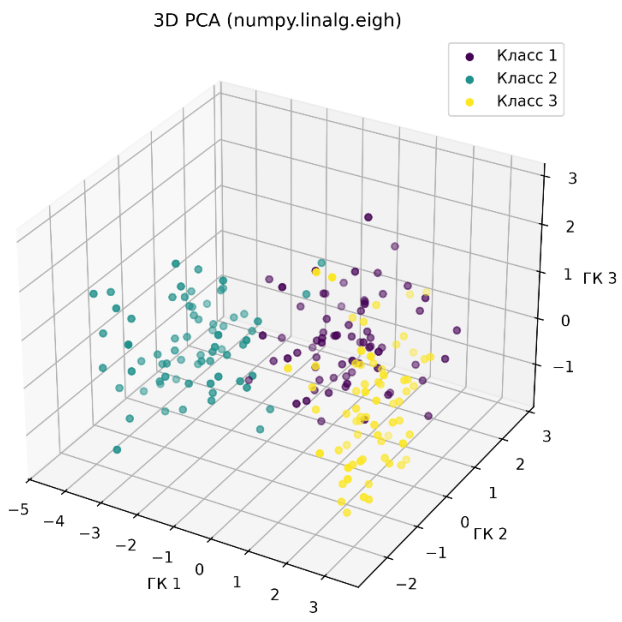
print(f"Потери при 2 компонентах: {loss_2 * 100:.2f}%")
print(f"Потери при 3 компонентах: {loss_3 * 100:.2f}%")

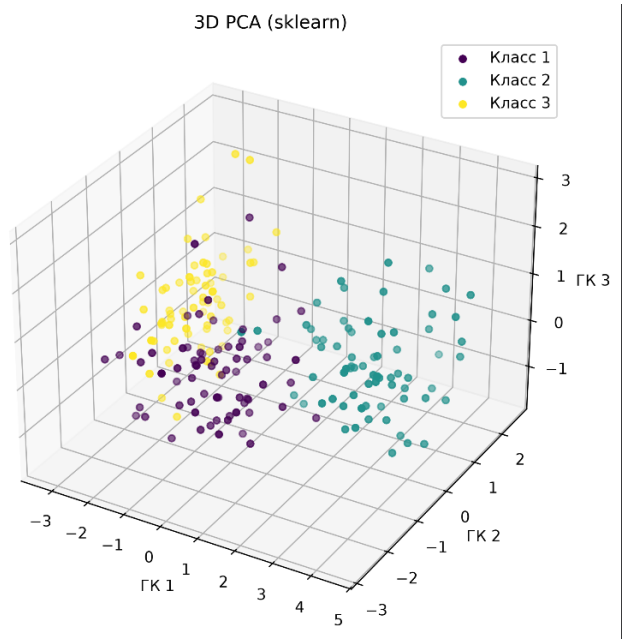
print(f"Объясненная дисперсия (2 компоненты): {sum(pca_2.explained_variance_ratio_) * 100:.2f}%")
print(f"Объясненная дисперсия (3 компоненты): {sum(pca_3.explained_variance_ratio_) * 100:.2f}%")
```

Результат работы программы:









Вывод: научился применять метод PCA для осуществления визуализации данных.